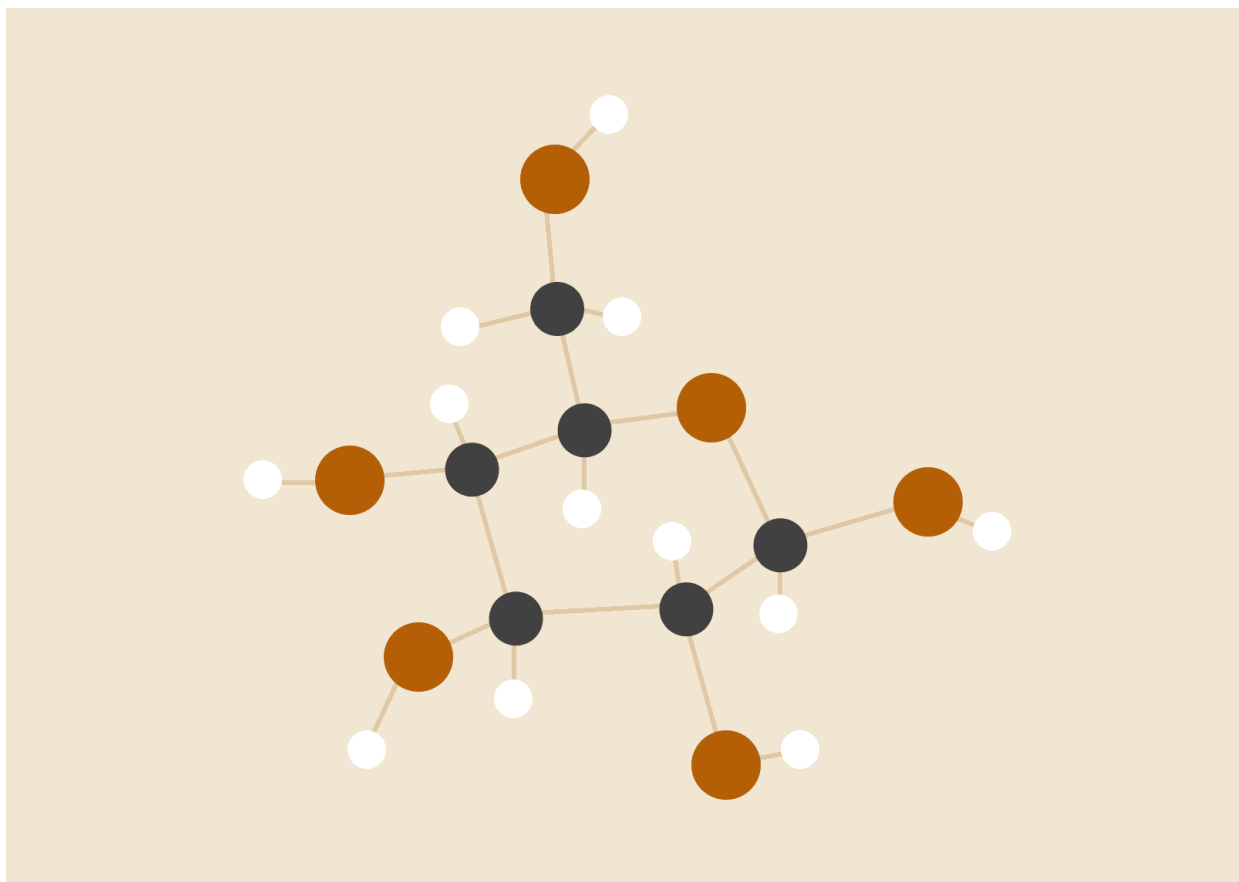


RAPPORT DU PROJET CO2K

L'application web d'éco-nutrition combinant suivi calorique et empreinte carbone



**Bounif Aksil | Amedjkane Amar | Laamouri
Abderrafie | Smail Elias | Chtioui Ahmed**

22/06/2026
MASTER 1 – MIAGE

Introduction et Analyse Fonctionnelle.....	4
Contexte : L'alimentation écologique.....	4
Problématique.....	4
Cahier des charges et objectifs du projet.....	4
Parcours utilisateur et Dashboard.....	5
Page d'accueil.....	5
L'Inscription et la Connexion.....	5
Le Dashboard.....	6
Espace Menu (Côté Nutrition).....	6
La section Profil.....	6
La section Planning.....	6
La section Panel de Gestion.....	7
Espace Communauté (Côté Social).....	7
La section Mon Profil.....	7
La section Explorer.....	7
Conception Algorithmique et Métier (Création de repas).....	7
Le socle commun : Le système de templates.....	7
Le générateur IA : Création rapide.....	8
Le filtrage par régime.....	8
L'adaptation aux métabolismes extrêmes.....	8
La simplification des courses.....	9
Le Constructeur Manuel : Choix et contraintes.....	9
La cohérence culinaire et les substitutions.....	9
Le solveur de grammage intelligent.....	9
L'empreinte carbone en temps réel.....	10
Complémentarité et dualité des modules.....	10
Architecture Technique et Choix Technologiques.....	10
Un monolithe unifié.....	10
Le Frontend.....	11
React et Vite.....	11
Shadcn/ui.....	11
TailwindCSS.....	11
Framer Motion.....	11
Le Backend.....	12
Express.....	12
La sécurité des accès.....	12
Supabase et Prisma.....	12
Hébergement cloud.....	13
La cascade des repas et portions.....	13

La flexibilité des programmes.....	13
Nettoyage automatique du fil social.....	14
Le système d'abonnements.....	14
Limitation des ressources.....	14
Typage et validation avec Zod.....	14
Validation à l'exécution avec Zod.....	14
Conteneurisation et Déploiement.....	16
L'environnement de développement local.....	16
L'orchestration des services.....	16
La synchronisation en temps réel.....	16
Le déploiement de production découplé (Vercel & Render).....	16
Déploiement du Frontend sur Vercel.....	17
Étape de Build.....	17
Étape d'Exécution.....	17
Déploiement du Backend sur Render.....	17
Étape de Build.....	17
Étape d'Exécution.....	17
La gestion des variables d'environnement.....	17
Variables hébergées sur Render (Backend).....	17
Variables hébergées sur Vercel (Frontend).....	18
Alignement avec les Principes de Génie Logiciel.....	18
Exploitation et Traitement de Données.....	18
Filtrage sélectif.....	19
Normalisation des données.....	19
Algorithmes d'Imputation des valeurs manquantes.....	19
Macronutriments manquants.....	19
Empreinte carbone manquante.....	19
Sucres et graisses saturées.....	19
Classification automatique en compartiments.....	20
Ingestion automatique.....	20
Stratégie de tests (Jest et Cypress).....	20
Les Tests Unitaires & d'Intégration Backend.....	20
Gestion des utilisateurs.....	20
Catalogue d'aliments.....	21
Gestion des Plannings.....	21
Programmes.....	21
Système communautaire.....	21
Les Tests de Bout en Bout (Cypress E2E).....	21
Inscription.....	22

Connexion.....	22
Profil.....	22
Mon Compte.....	22
Plannings.....	23
Panel de gestion.....	23
Communauté.....	23
Sécurisation de la livraison et réduction du coût de maintenance.....	23
Méthodologie et gestion de projet.....	24
Collaboration et flux de travail.....	24
Isolation des développements.....	24
Intégration et revues de code.....	24
Suivi de projet.....	25
Pratique du binomage.....	25
Partage des rôles.....	25
Réduction des bugs.....	25
Diffusion des compétences.....	25
Intégration des Patrons de Conception.....	26
Le Singleton.....	26
Problème.....	26
Solution.....	26
La Factory.....	26
Problème.....	26
Solution.....	26
La Façade.....	27
Problème.....	27
Solution.....	27
La Strategy.....	27
Problème.....	27
Solution.....	27
L'Observer.....	28
Problème.....	28
Solution.....	28
Conclusion.....	28
travail d'équipe.....	28
La rigueur du Génie Logiciel.....	29
La fierté d'un produit fini.....	29
Et pour la suite ?.....	29

Introduction et Analyse Fonctionnelle

Contexte : L'alimentation écologique

À une époque où la transition écologique mobilise de plus en plus de citoyens et où chacun s'efforce d'y mettre du sien pour réduire son empreinte carbone en empruntant les transports en commun, en privilégiant le vélo pour les trajets courts ou en préférant le reconditionné au neuf, certains secteurs restent encore peu documentés malgré leur impact majeur. C'est précisément le cas de l'alimentation. En effet, la production de nourriture engendre un coût écologique significatif qu'il s'agisse de son transport ou de sa transformation. L'assiette pèse ainsi lourd sur la balance environnementale en particulier à travers la consommation de viande liée aux émissions de l'élevage. Il devient donc indispensable d'évaluer l'impact environnemental de nos repas non seulement pour agir concrètement mais aussi pour éveiller les consciences tout en continuant à prendre soin de notre équilibre nutritionnel

Problématique

Dès lors, une question centrale se pose : comment sensibiliser la population au réel impact écologique de ses choix quotidiens notamment lorsqu'il s'agit d'un besoin vital comme l'alimentation afin de l'inciter à adopter une démarche active de compensation au jour le jour ?

C'est pour répondre à cet enjeu majeur de prise de conscience qu'est née l'idée de CO2K (combinant Cook et CO²). Plus qu'une simple application de suivi environnemental CO2K se positionne comme un véritable partenaire nutritionnel. Elle remplit de nombreux rôles à commencer par un accompagnement alimentaire sur mesure qui valorise la santé et la transition écologique. Ainsi la plateforme met à l'honneur une alimentation saine et éco-responsable en proposant des repas composés à 90 % d'aliments bruts et non transformés à l'exception des sauces

Cahier des charges et objectifs du projet

Le projet CO2K ne se limite pas à un simple outil technique de calcul il répond à un cahier des charges précis axé sur l'impact comportemental et l'accompagnement de l'utilisateur. Les objectifs majeurs du projet sont définis ainsi :

Rendre visible l'invisible : L'alimentation est un besoin vital mais besoin vital ne rime pas forcément avec naturel. Notre consommation est aujourd'hui industrialisée et mondialisée à tel point que nos aliments ont un réel impact sur notre planète. En prenant conscience de l'impact de ses besoins vitaux l'utilisateur comprendra que additionné à toutes ses consommations superflues le bilan écologique individuel devient rapidement insoutenable Chiffrer l'empreinte carbone de son assiette permet de créer un électrochoc et de responsabiliser le citoyen

Inciter à la compensation quotidienne : Puisqu'on ne peut pas arrêter de se nourrir, l'application utilise la donnée alimentaire comme un levier de motivation. En constatant le poids environnemental inévitable de ses repas, l'utilisateur est encouragé à équilibrer sa balance carbone personnelle au cours de sa journée en modifiant d'autres comportements

Promouvoir la santé brute : Le projet vise à démontrer que l'alimentation écologique va de pair avec la santé. L'objectif est de guider l'utilisateur vers des repas composés à 90 % d'aliments bruts (fruits, légumes, céréales, protéines), limitant ainsi l'impact industriel lié à l'ultra-transformation tout en préservant un apport nutritionnel de qualité (seules les sauces échappent à cette règle pour maintenir le plaisir culinaire)

Fournir un outil d'aide à la décision : Mettre à disposition une interface moderne, intuitive et interactive capable d'accompagner l'utilisateur dans sa transition en lui donnant le choix entre l'inspiration automatisée par l'IA et le contrôle manuel de sa consommation

Parcours utilisateur et Dashboard

Afin d'offrir un suivi personnalisé et sécurisé, l'application impose un certain parcours :

Page d'accueil

C'est la page sur laquelle l'utilisateur va apparaître au démarrage de l'application. Elle sert principalement à présenter l'application, ses objectifs et son rôle dans la transition écologique

L'Inscription et la Connexion

Pour accéder au dashboard l'utilisateur doit d'abord se créer un compte via un

formulaire d'inscription où lui sont demandés ses renseignements de base (nom, prénom, email, mot de passe) ainsi que ses paramètres physiologiques (taille, poids, sexe) afin d'adapter son profil nutritionnel. Une fois enregistré il peut s'authentifier sur la page de connexion

Le Dashboard

Une fois l'authentification réussie, l'utilisateur est redirigé vers son espace personnel. Ce Dashboard s'articule autour de cinq sections majeures réparties en deux espaces thématiques (Menu et Communauté) :

Espace Menu (Côté Nutrition)

La section Profil

Elle constitue la base de personnalisation de l'application en regroupant l'ensemble des données de l'utilisateur de manière centralisée. Elle prend en compte sa morphologie à travers des indicateurs physiques tels que le poids, la taille, l'âge et le genre. À partir de ces données, la plateforme calcule automatiquement ses objectifs nutritionnels définissant son apport calorique journalier idéal ainsi que ses besoins en macronutriments (protéines, lipides, glucides). Elle intègre également des seuils de vigilance à ne pas dépasser pour les nutriments sensibles (sucre, graisses saturées, sel). Enfin, ce profil propose un volet coaching pour adapter les recommandations au mode de vie (objectif physique, niveau d'activité quotidienne et préférences/restrictions éthiques comme les régimes sans porc ou sans viande)

La section Planning

Elle offre à l'utilisateur deux méthodes pour organiser son calendrier alimentaire de la semaine (découpé en quatre repas par jour). D'un côté, le générateur automatique conçoit instantanément un planning hebdomadaire complet. De l'autre, le compositeur manuel offre une flexibilité totale en permettant de construire ses journées repas par repas en distinguant aliments obligatoires et ingrédients non obligatoires

La section Panel de Gestion

Elle fait office de centre d'archivage et de contrôle où convergent tous les plannings hebdomadaires créés par l'utilisateur. Depuis cette interface il peut les consulter ou concevoir des programmes à plus long terme (cycles de 1 à 12 semaines de régime composés par une succession de plannings existants)

Espace Communauté (Côté Social)

La section Mon Profil

Elle offre à l'utilisateur une interface de publication dédiée au partage. Depuis cette page il peut créer des posts interactifs pour mettre en avant ses réussites et lier directement à sa publication un planning ou un programme complet issu de son Panel de Gestion. Une fois publiés ces posts sont partagés avec l'ensemble des utilisateurs qui peuvent y laisser des likes ou des commentaires

La section Explorer

Elle fait office de fil d'actualité global. L'utilisateur peut y consulter toutes les publications de la communauté, appliquer des filtres, rechercher des thèmes par mots-clés ou par créateur et s'abonner à d'autres profils pour regrouper leurs futurs partages dans son onglet dédié aux abonnements

Conception Algorithmique et Métier (Création de repas)

Le socle commun : Le système de templates

Pour concilier personnalisation et rapidité nous avons conçu une architecture à double entrée. D'un côté le Générateur IA permet de planifier sa semaine en quelques secondes. De l'autre le constructeur Manuel offre un contrôle total et précis sur chaque assiette

Les deux modules reposent sur une même base : la structure par templates. L'idée est simple : pour qu'un repas soit équilibré, il doit suivre une logique de composition. On divise donc l'assiette en plusieurs catégories d'aliments bruts (les féculents, les protéines, les légumes, les sauces, etc.) appelés bacs

Selon le moment de la journée le fonctionnement s'adapte :

Le matin et la collation se basent sur un template unique et fixe pour simplifier la composition

Le midi et le soir offrent plus de choix avec quatre modèles différents : un repas chaud classique, une salade, un sandwich ou un wrap. Choisir un repas chaud par exemple impose d'associer des féculents, une source de protéines, des légumes, et optionnellement un filet d'huile ou une sauce pour le plaisir et l'apport en lipides

Le générateur IA : Création rapide

Le but du générateur est d'aider l'utilisateur à planifier toute sa semaine en un clic à raison de quatre repas par jour tout en collant parfaitement à son profil métabolique

Pour composer les assiettes, notre algorithme ne pioche pas dans des plats préparés industriels. Il assemble des aliments bruts compatibles pour créer un vrai repas cohérent en procédant en trois étapes

Le filtrage par régime

Il élimine d'office les ingrédients incompatibles avec les restrictions de l'utilisateur (comme la viande pour un régime sans viande ou le porc pour un régime sans porc)

L'adaptation aux métabolismes extrêmes

Pour les profils ayant de très gros besoins énergétiques (au-delà de 4000 kcal pour les plus sportifs), l'algorithme applique des filtres spéciaux pour éviter que le volume physique de nourriture soit impossible à ingérer. Il privilégie des féculents très denses en glucides (comme le riz ou les pâtes) au détriment d'aliments volumineux mais moins caloriques. Côté protéines, il sélectionne des viandes ou alternatives très maigres pour éviter que les graisses saturées n'explorent avec les quantités et écarte les graisses saturées au profit d'acides gras insaturés

La simplification des courses

Pour éviter que l'utilisateur n'ait à acheter des dizaines d'ingrédients différents pour sa semaine, l'algorithme réutilise les aliments. Il duplique les deux repas les plus proches de l'objectif calorique d'un jour sur l'autre ce qui limite la liste de courses tout en gardant une excellente précision nutritionnelle

Le Constructeur Manuel : Choix et contraintes

Si le générateur IA permet d'aller vite, le constructeur manuel est le véritable cœur de CO2K. C'est ici que l'utilisateur conçoit ses repas au gramme près, réalise l'impact de chaque ingrédient sur son corps et sur la planète et apprend à faire les bonnes substitutions

Pour composer son plateau, on retrouve la même logique de templates, mais c'est cette fois l'utilisateur qui sélectionne son modèle de repas et choisit ses aliments dans les différents bacs. Plusieurs règles et calculs entrent alors en jeu pour l'accompagner :

La cohérence culinaire et les substitutions

Pour que les repas restent réalistes, notre moteur de règles (ReglesRepas) valide les choix en direct. Si on ajoute un ingrédient dans une catégorie déjà présente (remplacer du riz par des pâtes par exemple), le système remplace automatiquement l'ancien pour éviter les doublons absurdes. Le système gère aussi les restrictions de régime en masquant les aliments interdits et impose des associations logiques (comme ajouter un produit à tartiner si on sélectionne du pain le matin)

Le solveur de grammage intelligent

Une fois les aliments choisis, pas besoin de peser sa nourriture. Notre solveur (RepasSolver) tourne en arrière-plan et effectue jusqu'à 150 itérations pour trouver la portion idéale de chaque ingrédient. Il ajuste les poids pour coller au plus près aux calories et aux macronutriments (protéines, lipides, glucides) cibles du repas. De plus, il intègre des pénalités strictes : si l'assiette dépasse les limites de graisses saturées, de sel ou de sucre autorisées pour le profil de l'utilisateur, le solveur réduit automatiquement

les portions des ingrédients trop riches pour ramener le repas dans le vert

L'empreinte carbone en temps réel

À chaque fois qu'on ajoute ou remplace un aliment, l'application recalcule instantanément le bilan carbone du repas et de la journée. C'est là que réside l'aspect pédagogique de l'outil, en remplaçant simplement un steak de bœuf par du poulet ou des lentilles, l'utilisateur voit son score carbone chuter sur son écran alors que ses jauges de protéines et de calories restent équilibrées grâce au solveur. C'est le moyen idéal pour faire prendre conscience de l'impact écologique de nos assiettes

Complémentarité et dualité des modules

En résumé la force de CO2K repose sur la complémentarité de ses deux approches :

Le Générateur IA sert d'outil d'inspiration rapide et de structure globale. Il permet à l'utilisateur de démarrer sa semaine avec un squelette de repas équilibrés et adaptés à son métabolisme

Le constructeur Manuel offre quant à lui une précision chirurgicale. C'est l'outil d'ajustement au quotidien qui permet de substituer les aliments, d'affiner les portions grâce au solveur et de prendre conscience en temps réel de l'impact carbone de chaque ingrédient

Architecture Technique et Choix Technologiques

Un monolithe unifié

Pour développer CO2K, nous avons choisi de regrouper le Frontend et le Backend dans un seul et unique dépôt de code. C'est une architecture monolithique. Ce choix nous a grandement facilité la vie car en utilisant TypeScript des deux côtés nous pouvons partager instantanément nos modèles et nos types de données évitant ainsi les erreurs de communication entre le client et le serveur

En pratique l'application se divise en deux : le serveur Node.js qui expose une API REST et le client React qui est compilé de son côté. Pour le déploiement final nous avons

configuré des règles de redirection pour que toutes les routes du navigateur pointent vers notre fichier `index.html` principal

Le Frontend

Pour que l'expérience utilisateur soit la plus fluide possible nous avons basé notre interface sur quatre outils complémentaires :

React et Vite

React était indispensable pour gérer le dynamisme du tableau de bord. Dès qu'un utilisateur ajoute un aliment dans son assiette, les jauges de nutriments et le bilan carbone se mettent à jour instantanément sans recharger la page. Nous avons couplé React à Vite pour profiter d'un environnement de développement rapide notamment grâce au rechargement à chaud qui applique nos modifications de code en temps réel

Shadcn/ui

Pour concevoir rapidement une interface élégante et accessible nous avons intégré la bibliothèque de composants Shadcn/ui. C'est elle qui propulse les éléments clés de notre tableau de bord comme les modales de sélection, les barres de progression nutritionnelles, les menus déroulants, les formulaires de saisie ou les cartes. Ces composants sont entièrement personnalisables et s'intègrent directement dans notre code source

TailwindCSS

Pour le design et la mise en page Tailwind nous a permis de styliser nos composants Shadcn/ui et de coder rapidement une interface moderne et propre. C'est aussi grâce à Tailwind que nous avons pu intégrer facilement le mode sombre

Framer Motion

Afin de rendre l'application plus interactive et moins statique nous avons ajouté de petites animations lors de l'ouverture des modales de sélection d'aliments ou de

l'apparition des cartes de repas

Le Backend

Côté serveur nous avons besoin de rapidité pour faire tourner nos algorithmes de calcul de portions

Express

Nous avons créé notre API avec Express pour gérer toutes nos routes de manière asynchrone et propre. C'est ce serveur qui reçoit les demandes du frontend, exécute notre solveur mathématique et interroge la base de données

La sécurité des accès

Pour protéger les comptes de nos utilisateurs nous utilisons des jetons de sécurité (JWT). Quand un utilisateur s'identifie, le serveur lui renvoie un jeton que nous stockons dans le sessionStorage du navigateur. Nous avons préféré le sessionStorage au localStorage car les données y sont automatiquement supprimées dès que l'utilisateur ferme son onglet, limitant ainsi les risques de vol de session. Ce jeton est ensuite automatiquement envoyé dans les en-têtes de chaque requête pour valider l'accès aux pages protégées comme la sauvegarde des plannings. De plus nous avons configuré CORS pour que seul notre nom de domaine frontend soit autorisé à appeler l'API

Supabase et Prisma

Nous avons choisi une base de données relationnelle PostgreSQL hébergée dans le cloud grâce à la plateforme Supabase pour stocker nos données, le tout piloté par l'ORM Prisma. Voici comment nous avons structuré nos données pour répondre à nos contraintes

crée un programme de plusieurs semaines, nous avons créé une table intermédiaire CalendrierPlanning. Elle enregistre simplement l'ordre des semaines et lie un programme à un planning existant. On peut ainsi réutiliser le même planning plusieurs fois sans alourdir la base de données

Nettoyage automatique du fil social

Lorsqu'un utilisateur publie un post dans la communauté il peut y attacher un planning ou un programme. Pour éviter de se retrouver avec des posts communautaires vides ou cassés si l'utilisateur supprime ou modifie son planning d'origine nous avons configuré une suppression en cascade. Si le planning disparaît du panel de gestion le post associé est automatiquement nettoyé

Le système d'abonnements

Pour que les utilisateurs puissent s'abonner entre eux nous avons créé une table pivot Follow qui relie un utilisateur suiveur à un utilisateur suivi. Cela nous permet de filtrer le fil communautaire Abonnements en une seule requête SQL très simple et rapide

Limitation des ressources

Pour que notre base de données ne sature pas le serveur vérifie le nombre de plannings et de programmes d'un utilisateur avant de valider une sauvegarde. Si la limite académique de 10 est atteinte la requête est rejetée

Typage et validation avec Zod

Pour rendre notre code le plus sûr possible et éviter les comportements imprévus ou les injections de données corrompues, nous avons centralisé la gestion des données dans un fichier de type centralisé. Ce fichier assure une sécurité à la fois lors de la compilation et lors de l'exécution grâce à deux mécanismes complémentaires :

Validation à l'exécution avec Zod

Le simple fait de typer avec TypeScript ne protège pas contre les saisies invalides des utilisateurs lors de l'exécution (un âge négatif ou un format d'email incorrect). Pour y remédier nous utilisons la bibliothèque de validation Zod. Nous définissons des schémas stricts pour chaque flux de données:

- InscriptionFormSchema et LoginFormSchema valident le format des données de connexion
- ProfilFormSchema impose des limites physiques cohérentes (poids entre 30 et 120 kg, âge entre 15 et 100 ans)
- SavePlanningSchema et CreatePostSchema s'assurent que les plannings et posts partagés respectent la structure attendue avant d'être envoyés à l'API
- Typage statique de bout en bout et synchronisation Prisma : Nos formulaires et objets sont directement typés à partir des schémas Zod. Mieux encore nous lions ces types clients à ceux générés automatiquement par Prisma depuis notre base de données
- Assertions de typage préventives : Pour s'assurer que notre client React ne dérive jamais des modèles de notre base de données PostgreSQL nous avons écrit des assertions statiques dans le code telles que :
 - `const _checkInscr: Prisma.UtilisateurCreateInput = {} as InscriptionData;`
 - `const _checkProfil: Prisma.UtilisateurUpdateInput = {} as ProfilData;`

Ces lignes n'ont aucun impact en production mais elles agissent comme des garde-fous pour nous. Si nous modifions la base de données PostgreSQL (par exemple en renommant ou supprimant un champ de l'utilisateur), le compilateur TypeScript lèvera immédiatement une erreur au moment du build car le type du formulaire ne

correspondra plus à ce que la base de données attend.

Conteneurisation et Déploiement

L'environnement de développement local

Pour travailler dans un environnement reproductible et identique sur toutes nos machines de développement nous avons conteneurisé l'environnement sous un projet d'orchestration Docker

L'orchestration des services

Notre fichier `compose.yaml` démarre deux conteneurs distincts en parallèle, le frontend (port 5173) et le backend (port 3000)

La synchronisation en temps réel

Pour ne pas avoir à reconstruire manuellement nos conteneurs à chaque modification de code nous avons configuré l'outil `watch`. Docker surveille nos fichiers locaux pour synchroniser le code modifié dans les conteneurs, redémarrer le serveur si `server.ts` change, ou relancer un `npm install` interne si de nouvelles dépendances sont ajoutées

Le déploiement de production découplé (Vercel & Render)

Pour la production nous avons fait le choix de la simplicité et de la légèreté en n'utilisant pas Docker. À la place nous déployons directement l'application de manière découplée sur des plateformes (Vercel pour le frontend et Render pour le backend). Ce choix permet de maximiser les performances de l'application tout en restant dans le cadre des offres gratuites de ces hébergeurs

Les plateformes se connectent directement à notre dépôt Git et se chargent de compiler le code automatiquement lors de chaque commit

Déploiement du Frontend sur Vercel

Étape de Build

Vercel installe les dépendances du client et compile le frontend react pour générer les fichiers statiques HTML, CSS et JavaScript optimisés dans le dossier dist

Étape d'Exécution

Les fichiers générés sont distribués instantanément sur le réseau de Vercel. Le site se charge immédiatement pour l'utilisateur sans aucun temps de mise en veille

Déploiement du Backend sur Render

Étape de Build

Render installe les dépendances de l'API avec npm install et génère le client Prisma npx prisma generate nécessaire aux requêtes de la base de données

Étape d'Exécution

Render lance le serveur d'API Express via la commande npm run start qui exécute npx tsx [server.ts](#)

La gestion des variables d'environnement

Pour que l'application communique correctement entre le frontend, le serveur API et la base de données sans jamais exposer nos identifiants de sécurité en clair nous utilisons des variables d'environnement configurées de manière sécurisée sur nos plateformes respectives

Variables hébergées sur Render (Backend)

DATABASE_URL et DIRECT_URL : Elles stockent les chaînes de connexion vers notre base

de données Supabase

JWT_SECRET : Une clé privée utilisée par le serveur pour signer et vérifier l'authenticité des jetons de connexion des utilisateurs

CORS : Cette variable contient l'URL officielle de notre frontend déployé sur Vercel. Le serveur Express l'utilise pour interdire toute requête d'API non autorisée provenant d'un autre site web

Variables hébergées sur Vercel (Frontend)

VITE_URL : Elle indique à notre client React l'adresse exacte du serveur d'API de Render à contacter pour envoyer les requêtes (connexion, planification, etc.)

Alignement avec les Principes de Génie Logiciel

Exploitation et Traitement de Données

Conformément aux contraintes académiques de notre projet, CO2K repose sur des données réelles libres. Pour chiffrer l'impact carbone et nutritionnel de notre catalogue nous croisons deux bases de données publiques via leur code d'identification le Code CIQUAL :

- La table CIQUAL de l'ANSES : composition nutritionnelle des aliments (calories, protéines, lipides, glucides, sel, sucre, graisses saturées)
- La base AGRIBALYSE de l'ADEME : émissions de l'empreinte carbone en kg des aliments

Afin d'unifier et de préparer ces deux sources brutes pour notre application nous avons

mis en place un pipeline automatisé en Python (data/nettoyage.py) utilisant la bibliothèque Pandas

Filtrage sélectif

Le script élimine d'office les plats cuisinés complexes (pizzas, sandwichs industriels, desserts préparés) de la table CIQUAL pour isoler uniquement les aliments bruts, respectant ainsi notre cahier des charges d'une alimentation à 90 % brute

Normalisation des données

La fonction nettoyer_chiffre élimine les caractères textuels des valeurs (convertit les traces en valeurs numériques exploitables 0.0). La fonction nettoyer_nom standardise les chaînes de caractères, correction orthographique automatique comme oeufss en oeufs, passage en minuscules et suppression des parenthèses pour optimiser les recherches sur l'interface

Algorithmes d'Imputation des valeurs manquantes

Macronutriments manquants

Si les calories sont connues mais que des Macronutriments manquent, le script les calcule via la formule d'Atwater. S'il manque des données sur des aliments, le script injecte des valeurs représentatives de la catégorie

Empreinte carbone manquante

Pour les aliments n'ayant pas de correspondance directe, le script leur attribue la moyenne carbone calculée sur l'ensemble de leur sous-catégorie ou catégorie supérieure si la sous-catégorie est vide

Sucres et graisses saturées

Le script complète les valeurs nulles en estimant les quantités à partir de ratios observés

sur la même famille d'aliments

Classification automatique en compartiments

À l'aide de regex, le script classe automatiquement les aliments nettoyés dans l'un de nos 26 bacs métiers (PROT_VOLAILLE, RIZ, LEG, etc.) et configure leurs indicateurs métiers sous forme de booléen (estVege, estSandwich, estSnack)

Ingestion automatique

Le script se connecte à notre base PostgreSQL Supabase, applique une suppression en cascade et insère en bloc l'ensemble des aliments nettoyés prêts à l'utilisation

Stratégie de tests (Jest et Cypress)

Pour sécuriser notre application nous avons déployé une suite complète de tests divisée en deux catégories complémentaires : les tests unitaires / d'intégration back-end et les tests E2E

Les Tests Unitaires & d'Intégration Backend

Nos tests unitaires sont écrits avec le framework Jest. Afin d'isoler le code serveur de la base de données physique Supabase et de garantir des tests rapides indépendants de la connexion internet nous utilisons la bibliothèque jest-mock-extended pour effectuer un mock de notre base de données

Cinq suites de tests couvrent l'intégralité de nos fonctions d'accès aux données :

Gestion des utilisateurs

Vérifie la création de comptes (avec chiffrement du mot de passe), la récupération d'un utilisateur par son e-mail (getMail / getUtilisateurComplet) et la mise à jour des paramètres physiques du profil (majProfil). Teste également la remontée d'exceptions si la base de données échoue

Catalogue d'aliments

Valide la récupération de tous les aliments (getAllAliments), la récupération par identifiant unique (getAlimentById) et le filtrage brut des aliments par type de compartiment (getAlimentsParBac)

Gestion des Plannings

S'assure de l'enregistrement d'un planning (sauvegarderPlanning), de la récupération des plannings d'un utilisateur, de la modification des portions pesées en grammes (majPlanning), de la mise à jour du nom du planning et de la suppression d'un planning

Programmes

Valide la création de programmes multi-semaines (creerProgrammeComplet), la mise à jour de leurs informations, leur suppression et la réinitialisation de la planification

Système communautaire

Vérifie la création et la suppression de publications, l'ajout de commentaires, l'ajout/suppression des mentions J'aime (toggleLike) et le bon fonctionnement du système de suivi (toggleFollow) entre les utilisateurs

Les Tests de Bout en Bout (Cypress E2E)

Pour valider l'expérience utilisateur globale et le comportement du navigateur nous utilisons Cypress. Ces tests démarrent un navigateur automatisé localement afin de tester le frontend, Cypress utilise l'interception réseau pour mocker les réponses HTTP du serveur et forcer des scénarios d'erreurs ou de succès

Sept fichiers de test E2E couvrent l'intégralité du parcours utilisateur dans cypress :

Inscription

Vérifie la validation des formulaires vides (blocage et maintien de l'utilisateur sur la page d'inscription), la gestion des adresses email déjà utilisées (interception d'une erreur du backend et affichage d'un toast d'erreur) et la validation d'une inscription réussie en remplissant toutes les caractéristiques physiques (âge, taille, poids...), en interceptant le code de retour et en s'assurant que le formulaire est réinitialisé après confirmation

Connexion

Valide le comportement en cas de formulaire vide, l'échec de connexion avec de mauvais identifiants (interception d'un code 401 et affichage d'un message d'erreur sans redirection) et la réussite de la connexion (interception d'un JWT valide, stockage du token de sécurité dans le sessionStorage, notification de succès et redirection vers la page dashboard)

Profil

Teste l'affichage initial des besoins énergétiques et leur calcul instantané à l'écran lorsque les données physiques sont modifiées (le changement du poids calculant dynamiquement la valeur des calories affichées). Il valide également la sauvegarde des modifications en interceptant la requête PUT vers l'API utilisateur

Mon Compte

Gère l'espace personnel de l'utilisateur et son feed de publications. Il vérifie l'affichage de ses propres publications avec un compteur de posts, teste la création d'un nouveau post (remplissage du titre, contenu et sélection d'un programme associé via un menu déroulant), simule la suppression d'une publication (clic sur l'icône de corbeille, confirmation dans le toast, mise à jour dynamique du compteur) et valide l'affichage d'un état vide si l'utilisateur n'a encore rien publié

Plannings

Valide les fonctionnalités interactives du tableau de bord. Le test vérifie que les compteurs de calories et de bilan carbone commencent à zéro, simule le changement de jour actif (clic sur les boutons journaliers, vérification des styles CSS actifs et mise à jour de l'affichage de l'impact écologique), teste le processus de génération automatique d'un planning de repas par IA (interception de la route de sauvegarde et validation de la mise à jour des statistiques) et vérifie enfin le fonctionnement du bouton de copie (apparition du bouton coller) ainsi que le verrouillage par défaut du bouton de sauvegarde s'il n'y a pas de modification

Panel de gestion

Valide le centre de contrôle des programmes et plannings de l'utilisateur. Il s'assure que les programmes de musculation/nutrition et les plannings hebdomadaires créés s'affichent correctement dans leurs sections respectives. Il teste l'ouverture de la vue détaillée d'un planning avec recalcul en direct de ses apports en protéines, valide la suppression d'un programme avec confirmation par toast, et confirme l'affichage d'un écran d'avertissement lorsque le panel de l'utilisateur est vide

Communauté

Valide le feed et les interactions entre membres. Il teste l'accès au fil d'actualité montrant les publications des autres utilisateurs, le basculement vers l'onglet abonnements, la navigation vers la page de profil d'un autre membre, le fonctionnement du bouton d'abonnement/désabonnement dynamique (avec basculement des états d'affichage et interception des requêtes de suivi), ainsi que le retour fluide vers le flux social principal

Sécurisation de la livraison et réduction du coût de maintenance

Bien que l'écriture des tests unitaires Jest et de bout en bout Cypress ait été réalisée en fin de phase de développement pour consolider l'application avant sa livraison, cette démarche s'avère cruciale pour sécuriser le cycle de vie du projet. Elle permet de dresser

un bilan de santé global et automatisé du logiciel, garantissant qu'aucune régression fonctionnelle ne sera introduite lors des futures phases d'exploitation ou de maintenance évolutive

En amont de ces tests tardifs, la détection précoce des anomalies a tout de même été assurée durant l'implémentation grâce à l'utilisation systématique du typage strict avec TypeScript et des validations de schémas avec Zod. La combinaison de ces validations statiques au fil de l'eau et de la suite de tests dynamiques finale assure une base solide, réduisant significativement le coût global de correction des anomalies à long terme

Méthodologie et gestion de projet

Afin d'assurer une coordination efficace et de structurer le travail d'équipe nous avons adopté une méthodologie inspirée des démarches agiles. Cette approche itérative nous a permis de nous adapter rapidement aux défis techniques rencontrés, tout en maintenant un rythme de livraison régulier et incrémental des différentes fonctionnalités de la plateforme

Collaboration et flux de travail

L'historique de notre dépôt Git témoigne d'une répartition rigoureuse et transparente des tâches au sein de l'équipe. Pour structurer nos développements nous avons mis en place un flux de travail basé sur les principes du Feature Branching

Isolation des développements

Chaque fonctionnalité a été développée sur une branche dédiée (feature/profil, feature/Mon_Compte...) évitant ainsi de perturber la stabilité de la branche principale

Intégration et revues de code

La fusion des branches vers la branche principale master n'intervient qu'après une phase d'auto-évaluation et de revue de code. Cette étape permettait de vérifier la

cohérence du code, le respect des standards de typage et de style, et de résoudre d'éventuels conflits d'intégration

Suivi de projet

Ce versioning propre a également facilité le suivi de l'avancement global chaque commit traduisant une étape logique et documentée du développement

Pratique du binomage

L'implémentation de la logique métier la plus critique et complexe notamment le solveur mathématique de grammage (calcul des portions selon les objectifs caloriques) et le moteur de règles culinaires (gestion des régimes et exclusions alimentaires) a fait l'objet de sessions de programmation en binôme. Cette pratique a offert plusieurs avantages notables :

Partage des rôles

Pendant que l'un écrivait le code, l'autre prenait du recul pour concevoir l'architecture globale, anticiper les cas limites et vérifier la logique des algorithmes

Réduction des bugs

Le fait de croiser nos regards sur ces composants complexes a permis de détecter et de corriger des erreurs de logique mathématique ou des cas particuliers d'exclusions alimentaires dès la phase d'écriture plutôt que lors de phases ultérieures d'intégration

Diffusion des compétences

Ce travail collaboratif a favorisé un partage rapide de la connaissance technique et fonctionnelle du code rendant chaque membre du groupe autonome sur le fonctionnement interne de ces moteurs

Intégration des Patrons de Conception

Afin de concevoir une base de code modulaire facile à maintenir et conforme aux bonnes pratiques de programmation, nous avons implémenté plusieurs patrons de conception

Le Singleton

Problème

Ouvrir plusieurs connexions vers notre base de données Supabase à chaque requête d'API épuisent rapidement le pool de connexions du serveur Express

Solution

Dans db.ts, nous utilisons le patron Singleton sur notre instance de PrismaClient. L'application garantit qu'une seule et unique instance de connexion est créée et partagée tout au long du cycle de vie du serveur

La Factory

Problème

Instancier manuellement la structure de chaque repas (déjeuner, dîner, collation, petit-déjeuner) avec les filtres appropriés dans notre code de génération globale alourdit le contrôleur principal

Solution

Dans generator.ts, nous avons implémenté le patron factory à travers les objets GenerateurMatin, GenerateurPause et GenerateurPlats. Ils exposent tous une méthode commune .creer() qui retourne un repas complet et pesé, en encapsulant l'application des règles spécifiques à ce type de repas sans exposer le détail technique de leur

assemblage

La Façade

Problème

Le calcul de l'empreinte carbone et de la balance nutritionnelle implique de nombreuses formules (comme l'équation de Mifflin-St Jeor) et des manipulations complexes de tableaux de données. Éparpiller ces fonctions dans l'interface React ou les routes de l'API rendrait le code illisible

Solution

Nous avons créé une interface de Façade via l'objet `CalculateurImpact` dans `impact.ts`. Elle masque toute la complexité mathématique et relationnelle en fournissant des fonctions d'entrée/sortie simplifiées (`calculerStatsJournee`, `calculerBesoinsNutritionnels`) pour les composants d'interface et les routes du serveur Express

La Strategy

Problème

Les règles de cohérence culinaire, de filtrage d'aliments et de calcul de portions changent radicalement selon le type d'assiette sélectionnée (un repas chaud, une salade, un sandwich ou un wrap). Utiliser une suite infinie de conditions `if / else` serait ingérable

Solution

Nous avons séparé ces différents algorithmes de règles dans des modules distincts (`SectionRepas`, `SectionSandwich`, `SectionWrap`, `SectionSalade` dans `rules_repas.ts`). Lorsque l'application doit générer ou valider une portion, elle sélectionne dynamiquement l'objet de stratégie correspondant au template du repas et applique les méthodes associées

L'Observer

Problème

Lors d'actions comme la suppression d'un planning ou d'un programme qui entraîne la suppression en cascade des posts associés dans la communauté, l'application doit notifier l'utilisateur de manière non bloquante, sans coupler directement les pages métiers avec le composant graphique d'affichage des alertes

Solution

Nous utilisons le patron Observateur via notre système de notifications toasts. La page lance simplement une demande d'alerte. Un composant d'affichage central écoute ces demandes et affiche la boîte de dialogue à l'écran. Les pages n'ont ainsi pas à gérer le design ou l'état de l'alerte tout est centralisé

Conclusion

Mener à bien le projet CO2K a été pour nous une aventure à la fois stimulante et formatrice. Au-delà du défi technique, ce projet nous tenait particulièrement à coeur car il touche à un sujet concret qui nous concerne tous : comment concilier notre alimentation quotidienne, notre santé et notre impact sur la planète

Ce projet a été l'occasion de mettre en pratique et de consolider nos compétences de développeurs

travail d'équipe

L'adoption de méthodes de travail collaboratives comme le développement sur des branches Git dédiées et surtout les sessions de programmation en binôme, a été une vraie force. Travailler à deux sur les algorithmes les plus complexes, comme le solveur de portions ou le moteur de règles culinaires nous a permis de croiser nos idées, de résoudre les problèmes beaucoup plus vite et de partager nos connaissances au sein du groupe

La rigueur du Génie Logiciel

Concevoir cette application nous a montré l'intérêt concret des patrons de conception. Utiliser un Singleton pour notre base de données ou une Factory pour générer les repas a rendu notre code beaucoup plus propre et plus facile à maintenir. De plus, l'utilisation de TypeScript et Zod, complétée par nos tests Jest et Cypress à la fin du projet nous a donné confiance dans la stabilité de notre application

La fierté d'un produit fini

Voir tout notre pipeline fonctionner depuis le nettoyage des données avec Python jusqu'au déploiement de notre application sur Vercel et Render a été extrêmement gratifiant

Et pour la suite ?

Si nous devions poursuivre le développement de CO2K nous aimerions explorer d'autres pistes

- Connecter l'application à une intelligence artificielle pour proposer de vraies recettes de cuisine à partir des aliments générés
- Ajouter un système de liste de courses intelligente et collaborative pour faciliter la logistique des repas de la semaine
- Affiner l'aspect écologique en ajoutant d'autres indicateurs, comme la consommation d'eau ou la saisonnalité des aliments
- Apprendre en compte le prix pour ajouter un facteur de segmentation supplémentaire

En fin de compte, ce projet nous a prouvé qu'avec une bonne méthodologie et une conception rigoureuse, il est possible de créer un outil moderne, utile et agréable à utiliser tout en répondant à un véritable besoin citoyen